

Hybrid Cloud at Enterprise Scale

Private Kubernetes for Portability and Control



Shravani Gunturu

Senior Engineering Manager

SoFi (Galileo)

Why Hybrid Cloud & Private K8s

The Strategy: Deploy workloads across different providers—on-premises, public clouds, or at the edge. Enterprise hybrid cloud ensures consistent control, portability, and governance across all environments.

The Challenge

- Most end up **locked into one cloud**
- Regulatory, cost, and operational constraints
- Fragmentation hinders modernization

The Solution: Kubernetes

- Vendor-neutral abstraction layer
- De facto standard for orchestration
- Consistent deployment model everywhere

What “Private Kubernetes” Means

The Core Definition:

A private Kubernetes environment refers to dedicated clusters managed by the enterprise, ensuring full control over infrastructure and security policies.

Where It Runs

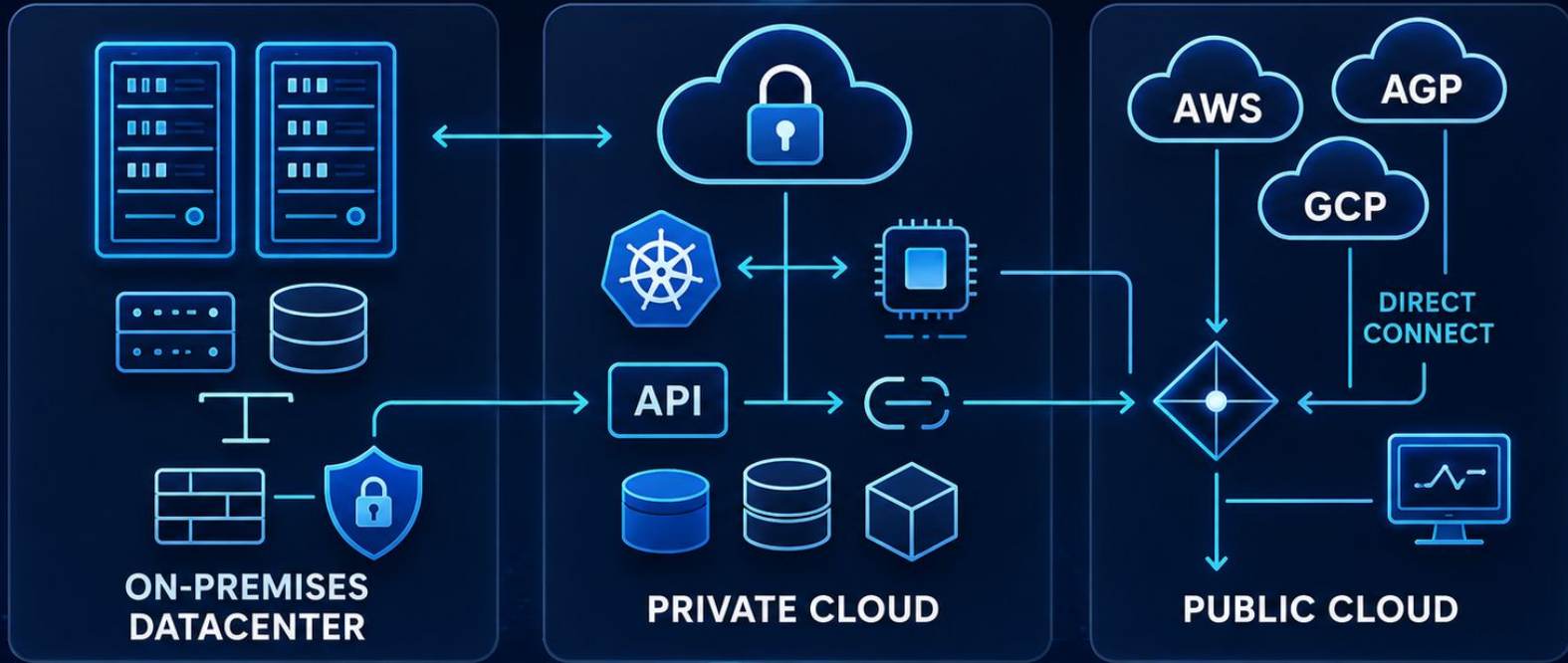
- On-prem data centers
- Private cloud infrastructure
- Isolated virtual networks (VPC/VNET) within public clouds

Common Distributions

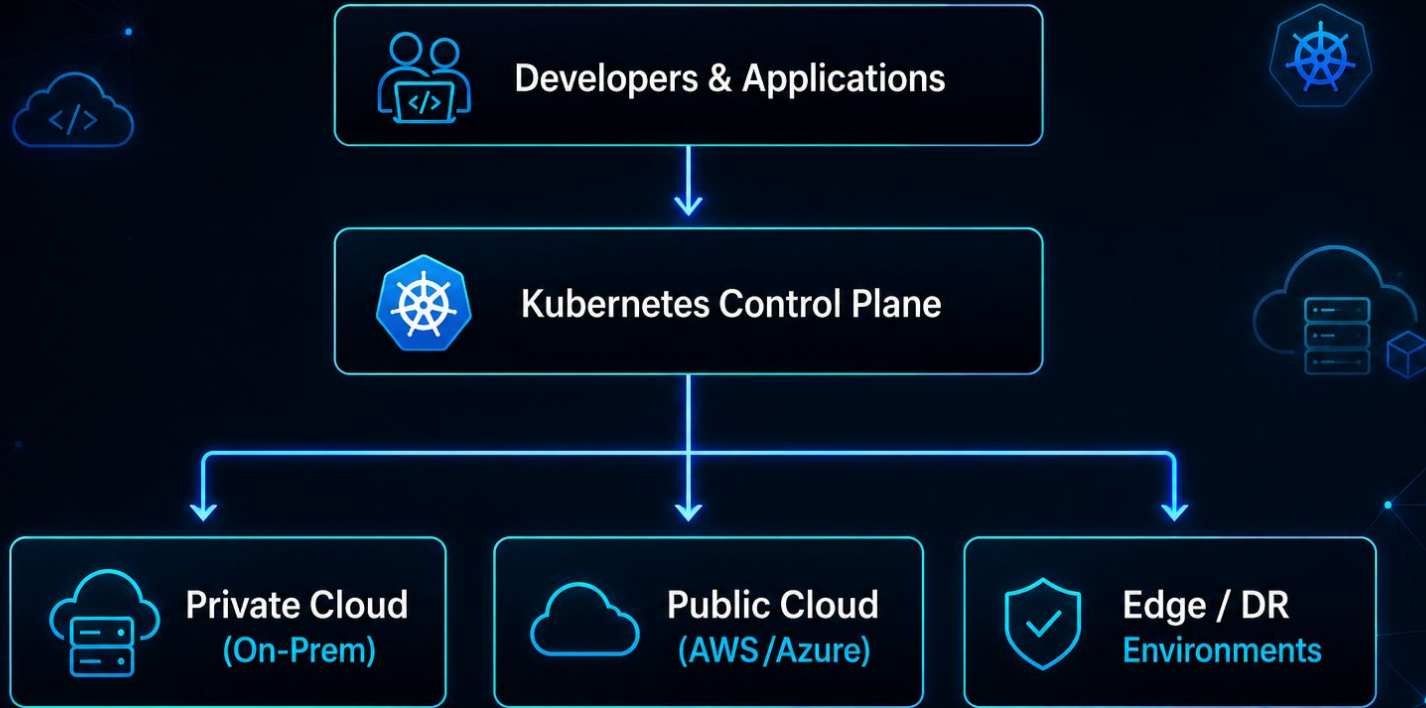
- Red Hat OpenShift
- VMware Tanzu
- Rancher



HYBRID CLOUD ARCHITECTURE



Core Value Proposition



Core Benefits

Portability

Avoid vendor lock-in by standardizing deployment and operations across environments.

Control & Compliance

Keep sensitive workloads and data in private environments while maintaining full regulatory oversight.

Consistency

Use the same tools, policies, and deployment pipelines everywhere for streamlined operations.

Optimization & Resilience

Place workloads where they perform best and cost least, ensuring business continuity.

Use Cases for Enterprises

Ideal Use Cases

- Regulated workloads & data sovereignty
- Legacy app modernization to cloud-native
- Disaster recovery & multi-region resilience
- Gradual migration with minimal refactoring

When This Approach Makes Sense

- Strict compliance or data sovereignty needs
- Multi-cloud portability requirements
- Large scale operations (100s of services)
- Strong internal platform engineering teams

Building a Private-Cloud K8s Platform

Developer Layer

→ Backstage + CI/CD

Service Layer

→ Istio mesh across clusters

Platform Layer

→ IAM + GitOps + Registry

Kubernetes Layer

→ kubeadm clusters, CNI + CSI

Data Center Layer

→ OpenStack / VMware
→ Ceph storage

Our Implementation

Private-Cloud K8s Platform

- Acts as a **true abstraction layer**
- Runs on **OpenStack and kubeadm**
- Extends portability to **AWS and Azure**
- Keeps the **developer experience identical** everywhere

Vendor-Neutral Capabilities

- Unified control plane patterns
- Standardized networking, IAM, and security
- Consistent cluster lifecycle tooling
- Pluggable infrastructure backends (OpenStack, bare metal, cloud)

Architecture Overview

Core Principles

- OpenStack for private-cloud compute, storage, networking
- kubeadm for lightweight, flexible clusters
- Custom controllers + operators for governance
- GitOps for cluster lifecycle
- Shared platform services: ingress, CNI, CSI, service mesh

Infrastructure Layer (Data Center Foundation)

Compute & OS

Hardware & Base OS

- Bare metal servers (Dell, HPE, etc.)
- Linux OS (Ubuntu, RHEL, Rocky)



Private Cloud

Virtualization Stack

- OpenStack (full IaaS control)
- Alternatives:
- VMware vSphere
- Proxmox VE



Storage Systems

Data Management

- Ceph (block + object)
- MinIO (S3-compatible)



Kubernetes & Networking Layers

Kubernetes Layer

Cluster Provisioning

- kubeadm (DIY, flexible)
- Cluster API (declarative clusters)

Enterprise Distributions

- Red Hat OpenShift
- Rancher



Networking Layer

CNI (Pod Networking)

- Calico (common, strong policies)
- Cilium (high-performance)

Ingress / Load Balancing

- NGINX
- HAProxy
- MetalLB



Why OpenStack and Kubeadm

OpenStack

- Cloud-like APIs without public cloud dependency
- Full control over cost, placement, and compliance
- Mature ecosystem for networking + storage
- Ideal for regulated workloads needing isolation

Kubeadm

- Lightweight, upstream-aligned Kubernetes
- Full lifecycle control
- Easy to replicate across environments
- Perfect for edge, dev, and specialized clusters

Infrastructure & Operations



IAM & Security

- ✓ Custom IAM integration
- ✓ Zero-trust principles
- ✓ Network segmentation
- ✓ Automated certificate + secret rotation
- ✓ Policy enforcement via OPA/Gatekeeper



Operational Tooling

- ⚙️ GitOps for cluster + app lifecycle
- ⚙️ Automated upgrades
- ⚙️ Drift detection
- ⚙️ Observability stack (metrics, logs, traces)
- ⚙️ SLOs + incident patterns



Identity & Security

Identity Integration

 Keycloak (OIDC provider)

 LDAP / Active Directory

Secrets Management

 HashiCorp Vault

Policy & Governance

 Open Policy Agent

 Kyverno

Container Security

 Trivy

 Falco



CI/CD, GitOps & Registry

CI/CD & GitOps

CI/CD Pipelines

- Jenkins
- GitLab

GitOps (Highly Recommended)

- Argo CD
- Flux



Container Registry

- Harbor (enterprise-grade, secure)
- Docker Registry





Observability & Service Mesh

Observability Stack

Metrics

- Prometheus

Visualization

- Grafana

Logging

- Elasticsearch
- Fluentd
- Kibana

Tracing

- Jaeger



Service Mesh (Hybrid Cloud)

Core Technology

- Istio

Capabilities Enabled

- Cross-cluster communication
- mTLS encryption (Zero Trust)
- Advanced traffic routing
- Multi-cloud failover





Why a Service Mesh? & Istio Architecture

The Problem Space

Challenges in Orchestration:

- Cross-cluster communication is hard
- Security inconsistent (No Zero Trust)
- Traffic routing limited

"Kubernetes gives you orchestration—but not a unified service-to-service communication layer across clusters."



Solution: Istio Architecture

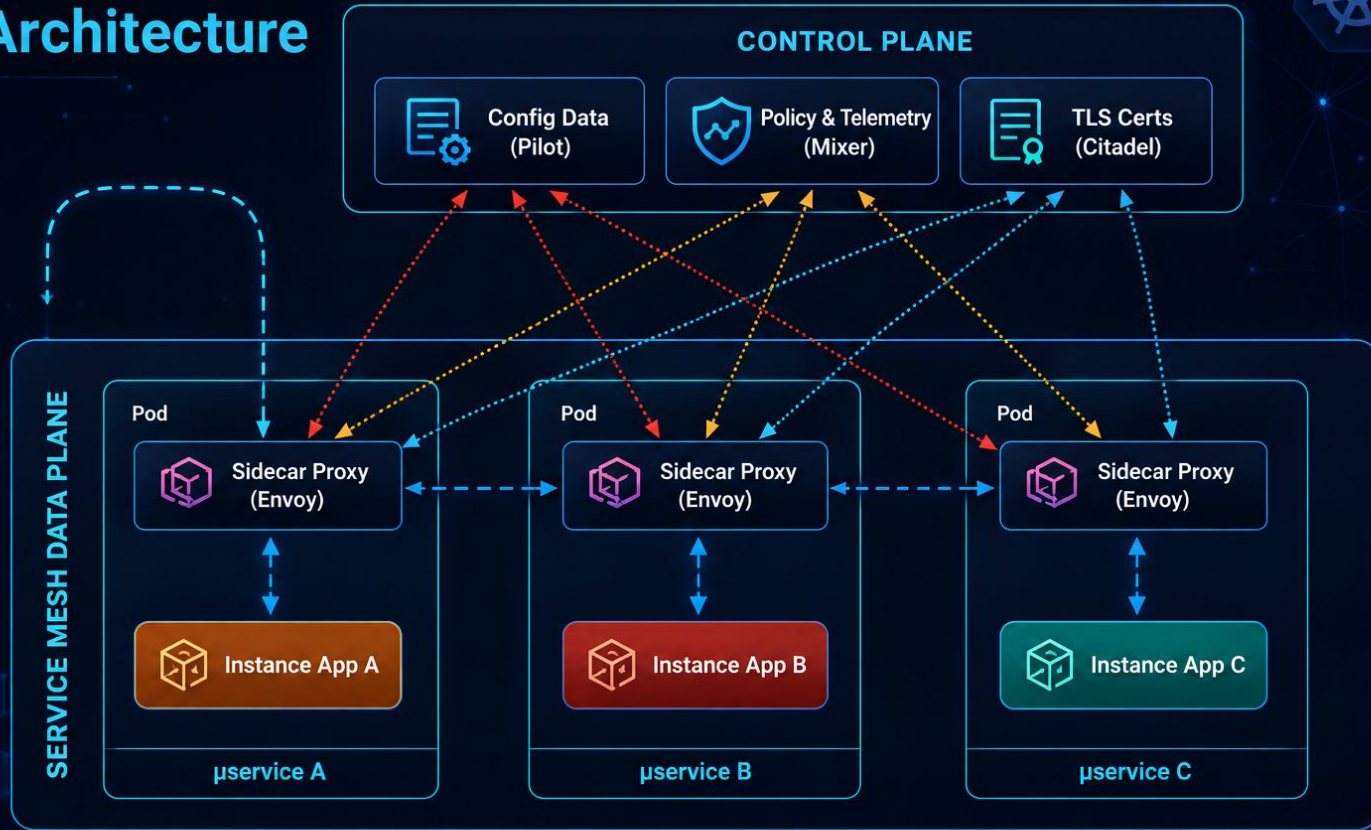
Key Components:

- Envoy sidecars
- Control plane
- Multi-cluster config

"Istio gives us a consistent data plane across all environments—private cloud and public cloud."



Istio Service Mesh Architecture





10. Platform Engineering & 11. Multi-Cluster

10. Platform Engineering Layer

Your "Secret Sauce"

You build:

- Internal developer platform (IDP)
- Self-service APIs
- Golden paths for deployment

Tools:

- Backstage (developer portal)
- Custom CLI / APIs
- Templates and scaffolding



11. Multi-Cluster Management

Key Technologies:

- Rancher
- Karmada
- Submariner





Best Practices



Adopt a Platform Engineering Model

Build an internal developer platform (IDP) to abstract Kubernetes complexity.



Use GitOps

Tools like Argo CD or Flux ensure consistent deployments.



Centralize Policy Management

Use policy engines (e.g., OPA/Gatekeeper) for governance across clusters.



Implement Service Mesh Carefully

Tools like Istio help with traffic management and security—but add complexity.



Invest in Observability

Full-stack visibility is essential for managing distributed systems at scale.



Outcomes & Lessons Learned

What This Enabled

- ✓ Reduced cloud dependency
- ✓ Improved security posture
- ✓ Predictable cost control
- ✓ Faster cluster provisioning
- ✓ Consistent governance across environments

Honest Lessons

- ⚠ Multi-cloud is not free — governance matters
- ⚠ Standardization is more important than tooling
- ⚠ Developer empathy drives adoption
- ⚠ Private cloud requires strong operational maturity
- ⚠ Abstraction layers must be intentional, not accidental

⚠ Key Challenges (and Reality Check)



Operational Complexity

Managing multiple clusters requires mature platform engineering.



Networking & Connectivity

Secure, low-latency communication is hard across regions.



Security Consistency

Unified identity and secrets management across clusters.



Skill Gaps

Steep learning curve for complex distributed infrastructure.

It may be overkill if:

- You're a small team
- Your workloads are simple
- You can fully operate within a single cloud



When This Approach Makes Sense

Private Kubernetes in hybrid cloud is ideal when:

Compliance & Sovereignty

You have strict compliance or data sovereignty needs

Multi-Cloud Portability

You want multi-cloud portability

Large Scale Operations

You operate at large scale (hundreds of services)

Engineering Expertise

You have (or can build) strong platform engineering teams

Thank You
Shravani Gunturu